

Checkers – Java

Rapport final

RANDRIAMIHAJA Latiana
RAMANANAHERISOA Fy-Tommy
HOAREAU Faniry
ATWI Daniel
RAZAFINDRAKOTO Sarah

3 avril 2026

Introduction

1 Contexte et objectifs du projet

Le projet a pour objectif de concevoir et développer un logiciel permettant de jouer au jeu de dames conformément aux règles officielles. Le programme doit permettre à deux joueurs de s'affronter sur un damier, soit en mode local, soit à distance via un réseau.

Le logiciel garantit le respect strict des règles du jeu tout en proposant une expérience utilisateur claire, robuste et extensible. Il s'inscrit dans une démarche de développement logiciel complète, intégrant la qualité du code, la documentation, les tests et les performances.

1.1 Variantes supportées

Le jeu de dames se décline en plusieurs variantes internationales, que le programme supporte via une option de configuration du plateau. Le style anglo-saxon, appelé Draughts, se joue sur un plateau de 8×8 , chaque joueur disposant de 12 pions. Le jeu de dames international adopte quant à lui un plateau de 10×10 , avec 20 pions par joueur. Enfin, les dames canadiennes étendent encore la surface de jeu avec un plateau de 12×12 , portant le nombre de pions à 30 par joueur.

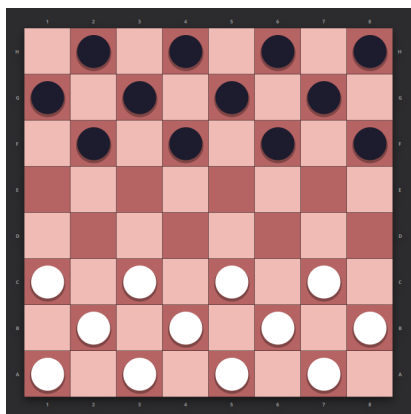


FIGURE 1 – Plateau 8×8

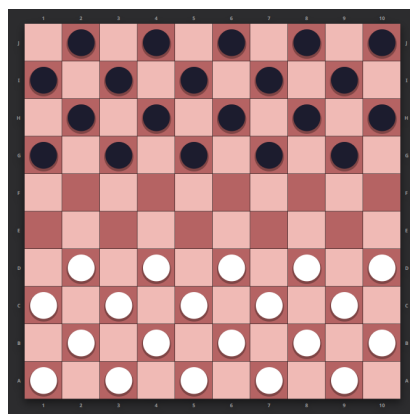


FIGURE 2 – Plateau 10×10

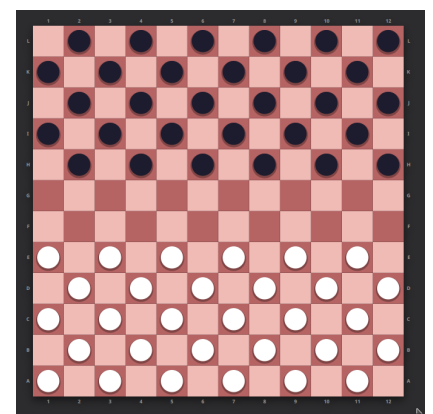


FIGURE 3 – Plateau 12×12

1.2 Règles du jeu

1.2.1 Mise en place et déroulement

Le damier est orienté avec une case foncée en bas à gauche. Les pions se placent exclusivement sur les cases foncées des quatre premières rangées de chaque camp. Les joueurs jouent chacun à leur tour ; les blancs commencent toujours.

1.2.2 Déplacements

Un pion se déplace d'une case vers l'avant en diagonale. Lorsqu'il atteint la dernière rangée adverse et s'y arrête, il est promu en dame, ce qui se matérialise visuellement par l'ajout d'un point jaune. La dame bénéficie alors d'une mobilité bien supérieure : elle peut parcourir librement une diagonale entière, d'autant de cases qu'elle le souhaite, aussi bien vers l'avant que vers l'arrière.

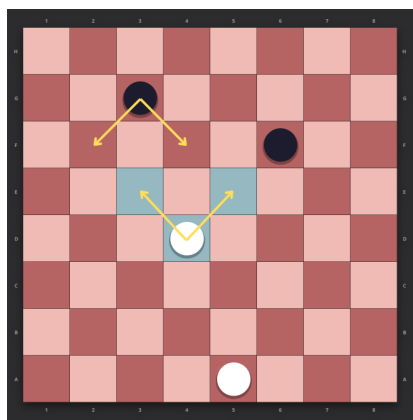


FIGURE 4 – Déplacement d'un pion

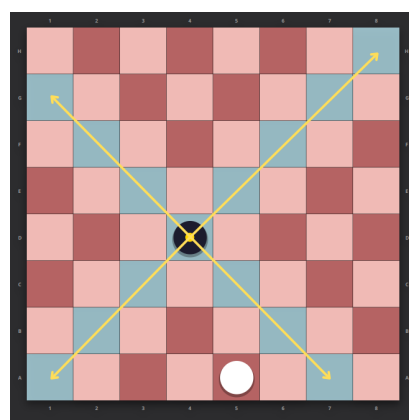


FIGURE 5 – Déplacement d'une dame

1.2.3 La prise par un pion

Un pion capture un pion adverse en sautant par-dessus lui pour se poser sur la case vide située immédiatement derrière. La prise peut s'effectuer vers l'avant comme vers l'arrière.

La prise est obligatoire. De plus, si après une capture le pion se retrouve à nouveau en position de prise, il doit continuer jusqu'à épuisement des captures possibles (rafle). Les pions capturés sont retirés du plateau uniquement à l'issue complète de la rafle, et non au fur et à mesure.

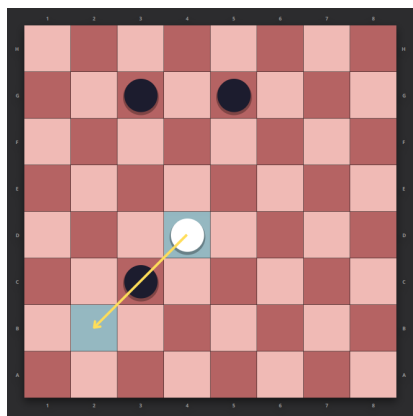


FIGURE 6 – Prise d'un pion

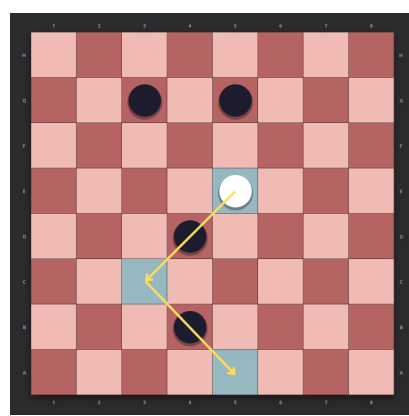


FIGURE 7 – Rafle

1.2.4 La prise majoritaire

Lorsque plusieurs séquences de capture sont possibles, le joueur est tenu de choisir celle qui capture le plus grand nombre de pièces, quel que soit leur rang. Ainsi, une séquence permettant de prendre deux pions est prioritaire sur la prise d'une dame isolée.

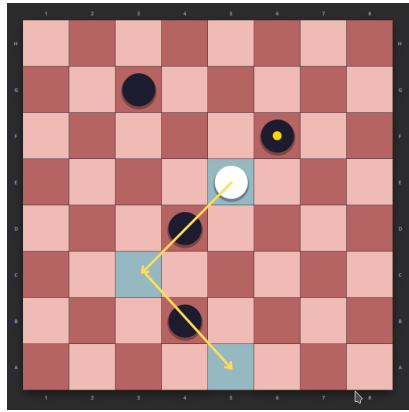


FIGURE 8 – Prise majoritaire : 2 pions vs 1 dam

1.2.5 La prise par la dame

Grâce à son amplitude de mouvement, la dame dispose de capacités de capture étendues. Elle doit prendre tout pion situé sur sa diagonale dès lors qu'une case libre est disponible immédiatement après. On ne peut passer qu'une seule fois sur un même pion, mais on peut traverser deux fois la même case.

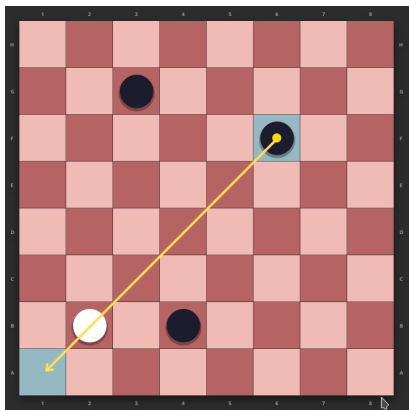


FIGURE 9 – Prise par la dame

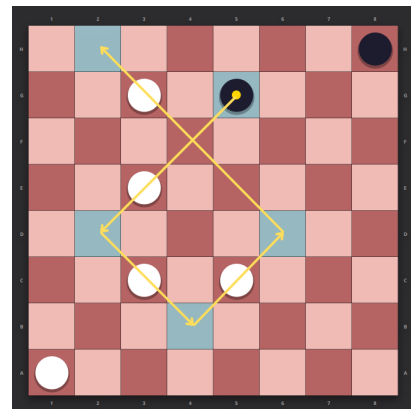


FIGURE 10 – Prise par la dame avec passage sur une même case

1.3 Conditions de fin de partie

Une partie prend fin dans l'un des cas suivants :

Victoire par capture. Un joueur remporte la partie en capturant l'intégralité des pièces adverses.

Victoire par blocage. Si un joueur ne peut effectuer aucun déplacement alors qu'il lui reste des pièces, il perd immédiatement la partie.

Nulle par accord de position. La partie est déclarée nulle si la même position se reproduit trois fois au cours de la partie (règle de la répétition triple).

Nulle par absence de progrès. Si aucune capture ni aucun mouvement de pion n'est effectué pendant 25 tours consécutifs (soit 50 demi-coups), la partie est déclarée nulle.

Nulle en fin de partie. Lorsqu'un joueur ne dispose plus que d'une seule pièce, si aucune issue décisive n'est atteinte en 16 tours (soit 32 demi-coups), la partie est déclarée nulle faute de matériel suffisant.

Nulle par blocage mutuel. Dans le cas exceptionnel où aucun des deux joueurs ne peut effectuer de mouvement, la partie est également déclarée nulle.

1.4 Versions existantes

Dans le cadre de notre phase d'analyse, nous avons étudié deux projets open-source de jeu de dames afin d'identifier des approches architecturales et algorithmiques pertinentes pour notre développement.

AwsDame

AwsDame [2] est une plateforme de jeu de dames multijoueur en temps réel développée dans un environnement JavaScript/Node.js. Bien que la pile technologique diffère de la nôtre (Java), son architecture réseau orientée événements offre des points de comparaison instructifs.

En particulier, ce projet adopte une logique de *matchmaking automatique* entre joueurs connectés, une idée que nous avons reprise dans notre propre serveur TCP. Notre classe `GameServer` implémente en effet une méthode `tryAutoStart()` qui détecte automatiquement la disponibilité de deux joueurs inactifs et déclenche une session sans intervention manuelle :

```
// GameServer.java -- démarrage automatique d'une partie
List<PlayerSession> idlePlayers = registry.getAllPlayers().stream()
    .filter(PlayerSession::isIdle)
    .collect(Collectors.toList());

if (idlePlayers.size() >= 2) {
    List<PlayerSession> pair = idlePlayers.subList(0, 2);
    GameSession session = registry.createSession(pair, sessionController);
    // notification GAME_START envoyée aux deux joueurs
}
```

Contrairement à AwsDame qui s'appuie sur WebSockets, notre solution repose sur des sockets TCP bruts avec un protocole textuel simple (REGISTER, MOVE, GAME_OVER, etc.), ce qui facilite les tests en ligne de commande et l'intégration avec un client Java natif.

Raven Checkers

Raven Checkers [1] est un programme de dames open-source développé en Python, conçu à des fins pédagogiques pour l'étude des tactiques de jeu et des algorithmes d'intelligence artificielle. Il couple la fonction d'évaluation de *Simple Checkers* aux algorithmes de recherche du projet AIMA [3].

L'analyse de ce projet a principalement influencé notre implémentation des algorithmes d'IA. Raven Checkers propose un MinMax classique sans gestion du temps, ce qui le rend peu adapté à des contraintes de jeu réel. Notre implémentation y répond de trois façons :

1. **Approfondissement itératif.** Plutôt que de fixer une profondeur statique, nos classes `MinMax` et `MinMaxAlphaBeta` explorent l'arbre niveau par niveau, de la profondeur 1 jusqu'à `maxDepth`, et conservent uniquement le résultat du dernier niveau entièrement exploré. Si le budget temps expire en cours de niveau, l'exploration partielle est abandonnée via une exception interne `TimeExceededException`, garantissant qu'aucun résultat incomplet n'est retenu.
2. **Élagage alpha-bêta.** La classe `MinMaxAlphaBeta` maintient deux bornes α (meilleur score garanti pour le maximisant) et β (meilleur score garanti pour le minimisant). Dès que $\beta \leq \alpha$, les branches restantes sont élaguées, réduisant significativement le nombre de nœuds évalués par rapport au MinMax pur.
3. **MCTS avec mode de sélection configurable.** Au-delà du MinMax, notre classe `Mcts` implémente la méthode de Monte Carlo Tree Search, avec un mode de sélection paramétrable (UCT ou ML) défini par l'énumération `SelectionMode`. Cette flexibilité est absente de Raven Checkers, qui ne propose qu'un seul algorithme de recherche.

La fabrique `Ai.buildAi(Configuration)` centralise le choix de l'algorithme à l'exécution :

```
// Ai.java -- sélection de l'algorithme via la configuration
switch (aiMode.toLowerCase()) {
    case "minimax"    -> return new MinMax(depth, timeMs);
    case "alphabeta"  -> return new MinMaxAlphaBeta(depth, timeMs);
    case "mcts"       -> {
        Mcts mcts = new Mcts(depth, timeMs);
        mcts.setSelectionMode(cfg.getSelectionMode());
        return mcts;
    }
}
```

Bilan comparatif

Le tableau 1 synthétise les principales différences entre les deux projets analysés et notre propre implémentation.

TABLE 1 – Comparaison des projets existants et de notre implémentation

Critère	AwsDame	Raven Checkers	Notre projet
Langage	JavaScript	Python	Java
Transport réseau	WebSocket	—	TCP (sockets)
Matchmaking automatique	Oui	—	Oui
Suivi des scores	—	—	Oui (W/L/D)
MinMax	—	Oui	Oui
Élagage alpha-bêta	—	Non	Oui
Approfondissement itératif	—	Non	Oui
MCTS	—	Non	Oui
Contrainte de temps pour l'IA	—	Non	Oui (configurable)

L'étude de ces deux projets nous a permis de valider nos choix de conception et d'identifier les lacunes à combler : gestion du temps dans la recherche adversariale, richesse algorithmique de l'IA, et protocole réseau adapté à un client Java interactif.

2 Algorithme et Structure de données

2.1 Bitboard

Nous utilisons des bitboards pour représenter l'état du plateau. Cette approche permet de représenter l'état du plateau sous forme d'entiers primitifs et de calculer les coups de manière globale via des opérations bit à bit, offrant des performances bien supérieures aux matrices 2D classiques.

2.1.1 Représentation et Numérotation des cases

Sur les 100 cases d'un plateau standard (10×10), seules les 50 cases noires sont utilisées. Nous utilisons la **notation de Manoury** pour numéroter ces cases actives de 0 à 49, de bas en haut et de gauche à droite.

L'état complet du jeu est encodé sur quelques entiers de 64 bits (`long` en Java), où chaque bit correspond à une case active (1 si la case est occupée, 0 si elle est vide). Cela nous évite d'écrire des formules mathématiques complexes pour définir l'état du plateau : tout repose sur l'indexation binaire.

.	45	.	46	.	47	.	48	.	49
40	.	41	.	42	.	43	.	44	.
.	35	.	36	.	37	.	38	.	39
30	.	31	.	32	.	33	.	34	.
.	25	.	26	.	27	.	28	.	29
20	.	21	.	22	.	23	.	24	.
.	15	.	16	.	17	.	18	.	19
10	.	11	.	12	.	13	.	14	.
.	5	.	6	.	7	.	8	.	9
0	.	1	.	2	.	3	.	4	.

2.1.2 Gain de mémoire chiffré

L'intérêt premier du bitboard est la compression des données. Comparons cette approche avec une structure de données "classique" comme un tableau à deux dimensions en Java (`int[10][10]`). Un tableau 2D classique nécessite 100 cases, chacune stockant un entier de 32 bits, soit un total de 3200 bits pour représenter l'état du plateau. En revanche, avec les bitboards, nous représentons l'ensemble du plateau avec seulement 4 variables de type `long` (64 bits chacune) : une pour les pions blancs, une pour les pions noirs, un pour les dames blanches et un pour les dames noires. Cela représente seulement 256 bits au total.

Le gain en mémoire vive est d'un facteur 12, ce qui accélère drastiquement les copies de plateaux lors de la recherche de l'IA.

2.1.3 Application des opérations binaires

L'avantage majeur du bitboard n'est pas seulement le stockage, mais la manipulation du plateau via le processeur. Au lieu de parcourir le plateau case par case, nous utilisons :

- **ET logique** ($\wedge$) : Pour détecter les collisions (ex : croiser un masque de déplacement avec le bitboard des pièces adverses).
- **OU logique** ($\vee$) : Pour fusionner les bitboards (ex : Blancs $\vee$ Noirs) et obtenir la carte de toutes les cases occupées.
- **NON logique** ($\neg$) : Pour inverser la carte d'occupation et isoler instantanément toutes les cases vides du damier.
- **Décalage binaire** ($\ll, \gg$) : Pour traduire toutes les pièces d'un joueur dans une direction donnée en une seule instruction d'horloge.

2.2 Algorithmes de manipulation

L'intérêt de la section précédente prend tout son sens ici : la génération des coups légaux repose sur la translation temporelle des bits.

TABLE 2 – Comparaison des décalages binaires selon la taille du plateau

Taille Plateau	8x8 (32 bits)		10x10 (50 bits)		12x12 (72 bits)	
Transition Ligne	P $\rightarrow$ I	I $\rightarrow$ P	P $\rightarrow$ I	I $\rightarrow$ P	P $\rightarrow$ I	I $\rightarrow$ P
Haut-Gauche	+3	+4	+4	+5	+5	+6
Haut-Droite	+4	+5	+5	+6	+6	+7
Bas-Gauche	-5	-4	-6	-5	-7	-6
Bas-Droite	-4	-3	-5	-4	-6	-5

2.2.1 Déplacement et prise par un pion

Lorsqu'un pion se déplace, on décale l'ensemble de son bitboard d'une direction donnée. Si le bit "atterrit" sur une position valide (une case vide, vérifiée via une opération $\wedge$ sur le bitboard inversé $\neg(\text{Occupées})$), le coup est légal.

Exemple concret de prise : Pour qu'une capture soit possible, un pion doit sauter par-dessus un pion adverse et atterrir sur une case vide. Sur un plateau 10×10 , supposons un pion blanc en case 32 (ligne paire). S'il veut capturer un pion noir situé en haut à gauche (case 27, ligne impaire), le décalage pour atteindre le pion noir est de +5. Pour atterrir derrière ce pion, le saut continue vers la case 21 (ligne paire), nécessitant un nouveau décalage de +6.

Le décalage total pour cette prise est donc constant : $5 + 6 = 11$ bits. L'opération binaire simplifiée de cette validation ressemble à cela :

$$\text{CibleSaut} = (\text{PionBlanc} \gg 5) \wedge \text{PionNoir}$$

$$\text{AtterrissageValide} = (\text{CibleSaut} \gg 6) \wedge \neg(\text{Occupées})$$

2.2.2 Déplacement et prise par une dame

Contrairement au pion, la dame peut parcourir un nombre illimité de cases vides sur une diagonale. D'un point de vue binaire, un simple décalage ne suffit pas : il faut propager le bit sur toute la diagonale jusqu'à rencontrer un obstacle. C'est la technique du *sliding piece generation*.

Voici le pseudo-code illustrant la génération des coups possibles pour une dame :

```

Fonction GenererCoupsDame(bitboard_dame, direction):
    coups_possibles = 0
    // Premier decalage dans la direction souhaitee
    rayons = decaler(bitboard_dame, direction)

    // Tant qu'on se deplace sur des cases vides
    Tant que rayons != 0:
        // On valide les cases vides rencontrees comme coups possibles
        coups_possibles = coups_possibles OU (rayons ET cases_vides)

        // On continue la propagation uniquement depuis les cases vides
        // La propagation s'arrete si on touche une piece ou le bord
        rayons = decaler(rayons ET cases_vides, direction)

```

3 Architecture du projet

Voici un exemple de code en Python :

```
def hello_world():
    print("Hello, World!")
hello_world()
```

Un exemple d'image flottante (Fig. 11).

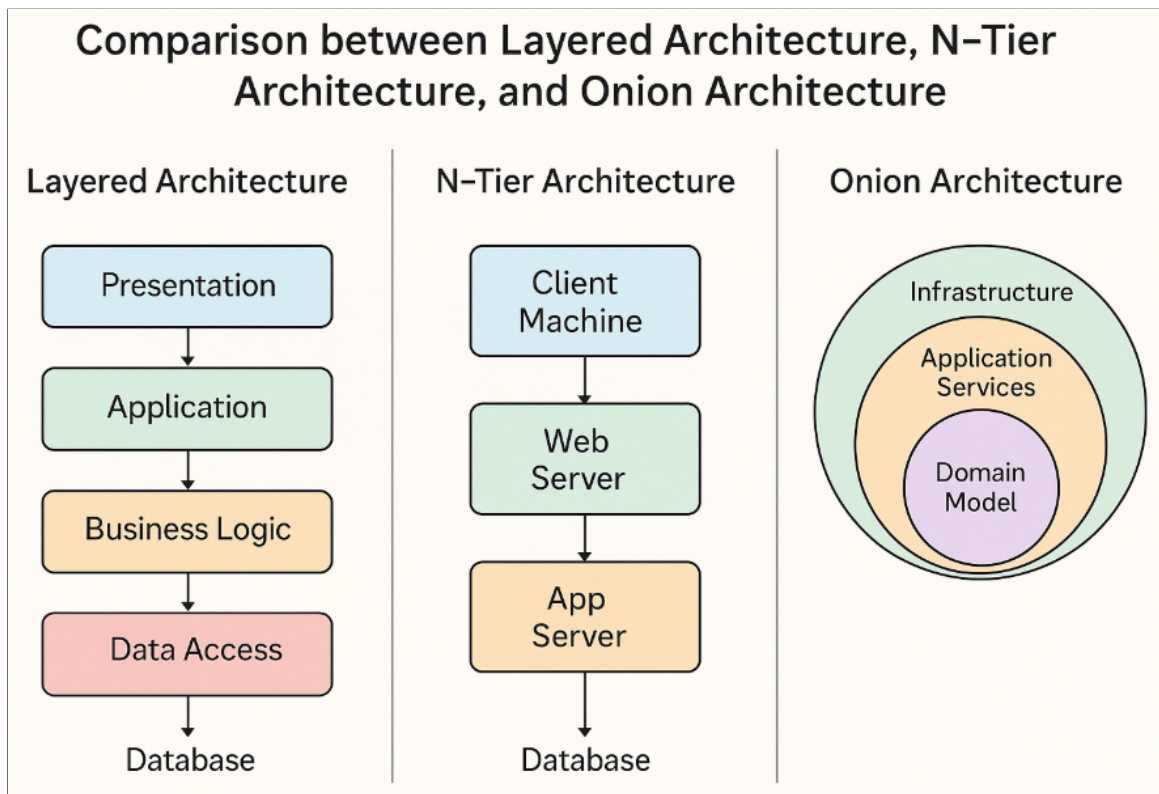


FIGURE 11 – Une architecture en couches

4 Explication approfondie

5 Revue critique de projet

5.1 Bug

5.2 Axes d'amélioration

Conclusion

Références

- [1] Brandon Corfman. Raven Checkers — open-source checkers with ai. <https://github.com/bcorfman/raven-checkers>, 2023. Consulté le 3 avril 2026.
- [2] mortexsa. AwsDame — multiplayer checkers platform. <https://github.com/mortexsa/AwsDame>, 2023. Consulté le 3 avril 2026.
- [3] Stuart Russell and Peter Norvig. *Artificial Intelligence : A Modern Approach*. Pearson, 4 edition, 2020.

A Liste des 40 besoins

TABLE 3 – État d’avancement des besoins fonctionnels

ID	Description du besoin	Statut
F1	Lancement du programme	
F2	Fichier de configuration	
F3	Internationalisation (i18n)	
F4	Interface en ligne de commande	
F5	Mode blitz	
F6	Mode contest	
F7	Interface graphique	
F8	Mode IA	
F9	Taille du damier	
F10	Gestion des règles	
F11	Gestion des erreurs de l'utilisateur	
F12	Gestion du undo/redo	
F13	Gestion des conseils	
F14	Gestion du blitz	
F15	Shell interactif	
F16	Proposer de sauvegarder avant de quitter	
F17	Édition de la commande en cours	
F18	Historique des commandes tapées	
F19	Complétion des commandes	
F20	Notation des coups	
F21	Sauvegarde et restauration d'une partie	
F22	Commentaires dans la sauvegarde	
F23	Format des paramètres du jeu	
F24	Format du damier	
F25	Format de l'historique	
F26	Module bitboard	
F27	Algorithmes de manipulation des bitboards	
F28	Menus de l'application	
F29	Raccourcis clavier	
F30	Drag'n'Drop	
F31	Temps de réflexion borné	
F32	Minimax avec $\alpha\beta$ -pruning	
F33	Fonctions d'évaluation pour Minimax	
F34	Approfondissement itératif de Minimax	
F35	Profondeur de recherche de Minimax	
F36	Monte Carlo Tree Search (MCTS)	
F37	Fonction de sélection par Machine Learning	
F38	Découverte de serveurs et connexion client-serveur basique	
F39	Serveur multi-clients et gestion de parties	
F40	Système d'invitations et gestion avancée des joueurs	